



「REACT JS」

CURRICULUM

12-Week React Training Curriculum

1. Course overview

1.1. Introduction

This comprehensive 12-week React training curriculum is designed to transform beginners into proficient React developers through a structured, hands-on learning approach.

1.2. What Trainees Will Learn

The program begins with essential JavaScript fundamentals, DOM manipulation, and development tools in the first week, ensuring students have a solid foundation before diving into React concepts.

The curriculum progressively introduces:

- React fundamentals
- Modern styling using Tailwind CSS
- Version Control
- TypeScript integration
- State management with Redux
- API integration
- Comprehensive testing strategies
- Professional deployment practices with CI/CD pipelines

The curriculum emphasizes modern development practices such as type safety with TypeScript, comprehensive testing with Jest and Cypress, automated deployment pipelines, and performance optimization.

1.3. Learning Approach

The first 9 weeks focus on individual learning and skill development through practical examples and progressively complex assignments. Each week builds upon previous knowledge while introducing new technologies and concepts.

1.4. Career Preparation

The program culminates with a comprehensive 3-week team project (Weeks 10–12), where students collaborate in groups to build a complete, production-ready application.

This phase simulates real-world development environments and emphasizes:

- Team collaboration
- Code reviews
- Project management
- Delivery of fully functional applications with proper documentation and deployment

1.5. Program Overview

By the end of this 12-week program, students will have both the technical skills and collaborative experience needed to excel in professional React development environments.



Week 1: JavaScript Foundations & Development Setup

Focus Areas

- Core JavaScript (ES6+)
- DOM Manipulation & Events
- Browser Storage (Local, Session, Cookies)
- Git & GitHub Version Control

Day 1–2: JavaScript Fundamentals

Objectives:

- Grasp modern JavaScript syntax and patterns
- Understand asynchronous programming using Promises and async/await
- Work with arrays and real-world data

1. Introduction to JavaScript

1.1. Introduction to JavaScript

1.1.1. What is JavaScript?

JavaScript is a **high-level, interpreted programming language** primarily used to **create interactive and dynamic web applications**. It runs directly in the browser and enables websites to respond to user actions without needing to reload the page.

What JavaScript can do:

- Change webpage content dynamically
- Handle user events like clicks or form submissions
- Validate forms before sending data
- Communicate with APIs and servers (fetch, AJAX)
- Power frameworks like **React**, **Vue**, and **Angular**
- Build full-stack applications with **Node.js**

1.1.2. Synchronous vs Asynchronous JavaScript

What is synchronous code?

Synchronous code runs line by line one task must finish before the next one begins.

Example:

```
js
console.log("Start");
console.log("Middle");
console.log("End");
```

What is asynchronous code?

Asynchronous code allows multiple tasks to run independently. If a task takes time (like fetching data), JavaScript won't wait for it — it moves on and handles the result later.

```
console.log("Start");

setTimeout(() => {
  console.log("Delayed task");
}, 1000);

console.log("End");
```

```
Start
End
Delayed task
```

1.1.3. Why is JavaScript asynchronous?

JavaScript runs in the browser's single-threaded environment, meaning it can only do one thing at a time. To remain fast and responsive, JavaScript uses asynchronous programming to handle long-running tasks (like API requests) without freezing the browser.

That's where **Promises** and **async/await** come in — helping us handle async code in a clean, readable way

1.1.4. Variable Declarations in JavaScript: var, let, and const

I. var – Old way (ES5 and before)

- Can be re-declared
- Function-scoped (**not block-scoped**)
- Can be hoisted (**declared after usage**)
- Generally, not recommended in **modern JavaScript**

```
var name = "John";
var name = "Alice"; // Allowed

if (true) {
  var greeting = "Hi"; // Accessible outside block
}

console.log(greeting); // "Hi"
```

II. let – Modern and safe

- Can be **reassigned**, but not re-declared in the same scope
- **Block-scoped** (only available inside {}) Can be hoisted (**declared after usage**)
- Preferred for variables that may change

```
let age = 25;
age = 26; // ✅

if (true) {
  let color = "blue";
}

// console.log(color); ❌ Error: color is not defined
```

III. **const** – Modern and safer

- Cannot be reassigned
- Block-scoped
- Used for constants or values that should not change

```
const PI = 3.14;
// PI = 3.14159; ❌ Error: Assignment to constant variable

const user = { name: "Sarah" };
user.name = "Alice"; // ✅ Object properties can change
```

1.1.5. ES6+ Features (with Real-World Context & Examples)

Modern JavaScript (ES6 and later) introduced powerful features that make code **cleaner, shorter, and more readable**. Here's a breakdown with real-life development examples.

1. Arrow Functions

Arrow functions provide a concise way to write functions and are especially useful for callbacks or inline functions.

Before (Traditional Function):

```
function greet(name) {
  return "Hello, " + name;
}
```

After (Arrow Function):

```
const greet = name => `Hello, ${name}`;
console.log(greet("Alice")); // Output: Hello, Alice
```

Real Use Case: In map or filter:

```
const users = ["John", "Jane", "Mike"];
const greetings = users.map(user => `Hi, ${user}!`);
console.log(greetings); // ["Hi, John!", "Hi, Jane!", "Hi, Mike!"]
```

2. Destructuring

Destructuring helps you extract values from objects or arrays in a simpler way.

```
const user = { name: "John", age: 25, role: "admin"};
```

// Instead of:

```
const name = user.name;
const age = user.age;
```

// Use destructuring:

```
const {name, age} = user;
console.log(name); // John
console.log(age); // 25
```

Real Use Case: Extracting API response data:

```
const response = {data: {id: 1, username: "john123"}};
const {data: {username}} = response;
console.log(username); // john123
```

3. Template Literals

Template literals make it easier to **embed variables and expressions into strings** using backticks (`), instead of concatenating with +.

```
const product = "Laptop";
const price = 1500;
console.log(`The price of the ${product} is ${price}`);
// Output: The price of the Laptop is $1500
```

```
const user = { name: "Sarah", email: "sarah@example.com" };

const html = `
  <div>
    <h2>${user.name}</h2>
    <p>Contact: ${user.email}</p>
  </div>
`;

document.body.innerHTML = html;
```

Day 3: DOM Manipulation & Event Handling

Objectives:

- Dynamically interact with the browser interface
- Learn to listen and respond to user actions

1. Selecting Elements & Handling Events

```
<button id="clickMe">Click Me</button>
<script>
  const button = document.getElementById('clickMe');
  button.addEventListener('click', () => {
    alert('Button was clicked!');
  });
</script>
```

2. Creating & Inserting Elements Dynamically

```
const container = document.querySelector('#app');
const newElement = document.createElement('p');
newElement.textContent = 'This is a new paragraph.';
container.appendChild(newElement);
```


Day 4: Browser Storage & APIs

Objectives:

- Understand different types of browser storage
- Learn when and why to use localStorage, sessionStorage, and cookies
- Practice storing and retrieving persistent data

1. **localStorage** – Long-term, persistent storage

Use Case: Save user preferences, saved tasks, or tokens across browser sessions(size(~5MB)).

```
const settings = { theme: 'dark', language: 'en' };
localStorage.setItem('settings', JSON.stringify(settings));

// Retrieving
const saved = JSON.parse(localStorage.getItem('settings'));
console.log(saved.theme); // Output: dark
```

● **Why use:** Data stays even if the user closes the browser(5MB per origin).

● **Avoid for:** Sensitive data (not encrypted), large datasets.

2. **SessionStorage** – Temporary storage tied to a single tab

Use Case: Store temporary session data like form progress or login session before confirmation(size(~5MB)).

```
sessionStorage.setItem('sessionToken', 'abc123');
```

```
// Retrieve
```

```
console.log(sessionStorage.getItem('sessionToken')); // Output: abc123
```

● **Why use:** Clears automatically when the tab is closed.

● **Avoid for:** Multi-tab usage or when data needs to persist across sessions.

3. **Cookies** – Small pieces of data sent with HTTP requests

Use Case: Store login tokens or user IDs that the server needs to access on each request.

```
document.cookie = "username=john; expires=Thu, 18 Dec 2024 12:00:00 UTC";
```

● **Why use:** Required when working with backend authentication (especially for server-side sessions).

● **Avoid for:** Large data (cookies are small, ~4KB), complex objects.

Day 5: Git & GitHub Essentials

Objectives:

- Understand version control basics
- Practice common Git commands
- Learn to collaborate using GitHub

1. Initialize Git Project

```
git init
git add .
git commit -m "Initial commit"
```

2. Work with Branches

```
git checkout -b feature/task-form
# Make changes
git add .
git commit -m "Add task form UI"
git checkout main
git merge feature/task-form
```

3. GitHub Workflow

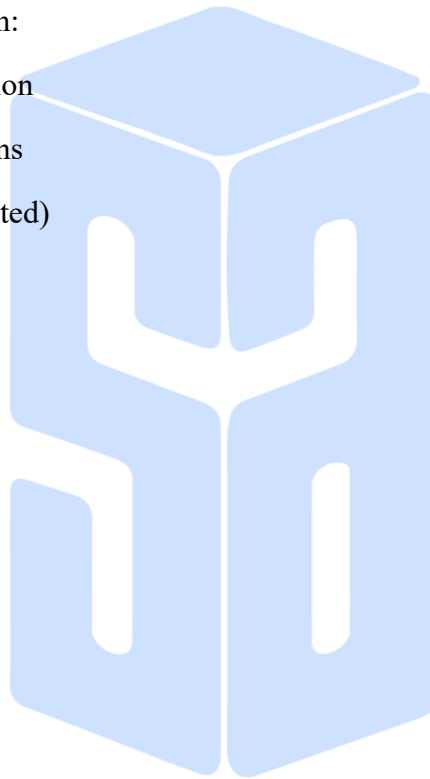
- Create a remote repo on GitHub
- Push your code:
 - `git remote add origin https://github.com/username/task-manager.git`
 - `git push -u origin main`
 - Open Pull Request and assign reviewers

Weekend Project: Personal Task Manager

Goal: Apply all concepts from the week.

Project Features:

- Add, Edit, Delete, and View tasks
- Store tasks using local Storage
- Responsive design with CSS Grid or Flexbox
- Host on GitHub with clear commit messages
- Add README file with:
 - Project description
 - Setup instructions
 - Live link (if hosted)



Week 2: React Introduction & Tailwind CSS

Focus:

Learn the basics of React how components work, how to pass data, how to render conditionally and apply Tailwind CSS to style components in a clean, responsive way

Day 1–3: React Fundamentals

1. What is React?

React is a JavaScript library for building user interfaces. It allows you to break a page into reusable components that manage their own state and update efficiently when data changes.

2. Components: Functional Components and JSX

Functional components are basic building blocks in React. JSX is the HTML-like syntax inside JavaScript. **Why it matters:** Components let you reuse code. Think of them like Lego blocks — you can mix and match them to build a UI.

```
const Button = () => (  
  <button className="bg-blue-500 text-white px-4 py-2 rounded">  
    Click me  
  </button>  
)
```

3. Props: Passing and Using Data in Components

What are Props in React?

Props (short for **properties**) are how you pass data from a **parent component** to a **child component**. Think of props like **function arguments** you provide values that the component will use.

Why Are Props Important?

- They make components **dynamic** and **reusable**
- They let parent components control what the child displays
- Help keep code **modular** and **organized**

Basic Example: A Reusable Card Component

```
jsx

const Card = ({ title, description }) => (
  <div className="p-4 border rounded shadow">
    <h2 className="text-xl font-bold">{title}</h2>
    <p>{description}</p>
  </div>
);

// Usage:
<Card title="React" description="A JavaScript library for building UIs" />
```

3.1.The children Prop

What is children in React?

children is a special prop that allows you to pass **JSX elements** between the opening and closing tags of a component.

When to use children:

- When the parent wants to wrap custom content
- For layout components like cards, modals, wrappers, sections

```
const Wrapper = ({ children }) => (
  <div className="border p-4">{children}</div>
);

// Usage:
<Wrapper>
  <h1>Hello World</h1>
  <p>This content is passed as children</p>
</Wrapper>
```

Pros and Cons of Using Props

Advantages:

- Encourages reusability and modularity

- Keeps components stateless and easy to test
- Improves readability by separating layout from data
- Works seamlessly with TypeScript (with interfaces)

Limitations:

- Too many props can make components hard to read or maintain
- Deep prop drilling (passing props through many levels) can become complex — use **Context** or **state management** for that
- Props are **read-only** — you can't change them from inside the component

4. Conditional Rendering

Render different components or UI based on conditions using:

- **&& for short conditional display**
- **ternary (? :) for full conditional logic**

Why it matters: Helps control what users see depending on their actions or app state.

Example:

```
{isLoggedIn ? <Dashboard /> : <Login />}
```

Short Form:

```
{showMessage && <p className="text-green-600">Success! </p>}
```

Day 4–5: Tailwind CSS Integration

What is Tailwind CSS?

Tailwind CSS is a modern utility-first CSS framework that lets you style your web pages by applying predefined CSS classes directly in your **HTML or JSX**.

Instead of writing custom CSS rules in external files (like .css or .scss), you build designs by combining small utility classes in your component

Why Tailwind CSS is Important

1. Faster Development

You don't have to write custom styles from scratch. You just combine utility classes to design components directly.

2. Clean & Maintainable Code

Tailwind encourages consistency. Instead of dozens of custom class names, you use a consistent system of utility classes that are predictable.

3. Responsive by Default

Tailwind has built-in responsive utilities (like md:, lg:) so you can easily design for all screen sizes.

4. Design Systems Without Custom CSS

You can create reusable components (e.g., PrimaryButton, Card, Badge) with consistent spacing, colors, and typography using only Tailwind classes — no extra CSS file needed.

5. Popular in the Industry

Tailwind is used by companies like GitHub, Shopify, and Netflix. It's widely adopted in modern frontend stacks including **React**, **Vue**, **Next.js**, and more.

1. Setup Tailwind in React

Install Tailwind and configure it:

```
bash

npm install -D tailwindcss
npx tailwindcss init
```

Update `tailwind.config.js` and add Tailwind's CSS to your app:

```
css

/* index.css */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

2. Utility-First Styling with Tailwind

Tailwind allows you to write styling directly in your component using class names.

Example:

jsx

```
<div className="bg-blue-500 text-white p-4 rounded shadow hover:bg-blue-700">  
  Tailwind Styled Box  
</div>
```

Responsive design:

jsx

```
<div className="w-full md:w-1/2 lg:w-1/3">  
  Responsive width  
</div>
```

Mini Project (End of Week 2): Weather Dashboard

Goal: Use everything learned to build a real component.

Features:

- Reusable Card component for weather info
- Grid layout using Tailwind (grid-cols-2, md:grid-cols-4)
- Conditional rendering (e.g., if no data, show a message)
- Styled with hover and responsive Tailwind classes

Week 3: React with TypeScript & State Management

Focus:

Enhance React with TypeScript for safety and clarity. Learn how to manage component state, fetch data, and create custom hooks and global state using context and reducers.

Day 1: TypeScript Recap / Introduction

What is TypeScript?

- TypeScript is a typed superset of JavaScript that adds static types to your code.
- It compiles down to JavaScript, so browsers can run it.
- It helps catch errors early and improves code readability, auto-completion, and developer confidence.

Why TypeScript in React?

- **Safer code** — you know what props, state, and data should look like
- **Better editor support** — IntelliSense, refactoring, auto-suggestions
- **Fewer bugs in large apps** — helps teams scale safely
- Works natively with React via interfaces and generics

Key TypeScript Concepts for React Devs

1. Variable Types

```
let name: string = "Alice";
let age: number = 30;
let isLoggedIn: boolean = true;
let tags: string[] = ["react", "typescript"];
```

2. Union Types and Optional Types

```
let id: number | string = 123;
let nickname?: string; // Optional
```

3. Interfaces vs Types

Interface (for objects and props):

```
ts

interface User {
  name: string;
  age: number;
}
```

Type (similar use, more flexible):

```
ts

type Status = "loading" | "success" | "error";
```

4. Functions with Types

```
function greet(name: string): string {
  return `Hello, ${name}`;
}

const sum = (a: number, b: number): number => a + b;
```

5. Type Assertion

Tell the compiler what type to assume:

```
const input = document.querySelector("input") as HTMLInputElement;
input.value = "Typed!";
```

Hands-On Tasks for Day 1

- ✓ Declare variables with types (string, number, boolean, arrays)
- ✓ Create a user interface and use it in an array
- ✓ Write a function that takes a typed object and returns a string
- ✓ Refactor a small JS function into TypeScript with full type safety
- ✓ Bonus: Try writing a generic `useLocalStorage<T>` function

Day 2: State Management with Hooks

What are React Hooks?

React Hooks are special functions that let you “hook into” React’s features — like **state**, **side effects**, and **context** — from within functional components.

Before hooks, React developers had to use **class components** to manage state and lifecycle methods. Hooks let you do all that in functional components, which are cleaner and easier to reuse.

Why Are Hooks Important?

- Make functional components powerful (state, lifecycle, shared logic)
- Eliminate the need for class components
- Simplify logic and encourage reuse with **custom hooks**
- Improve readability and maintainability

1. useState – Managing Component State

The useState hook lets you add state (data that changes) to your components.

Why use it?

To track dynamic values like:

- Form inputs
- Toggle states (e.g., dark mode)
- API data
- Count values

Example:

```
const [count, setCount] = useState(0);

return (
  <button onClick={() => setCount(count + 1)}>
    You clicked {count} times
  </button>
);
```

Explanation:

- **count** = current value
- **setCount** = function to update it
- State updates trigger re-rendering of the component

2. useEffect – Handling Side Effects

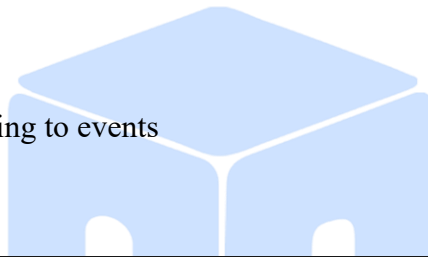
useEffect lets you run code after the component renders — perfect for side effects like:

- Fetching data
- Setting up event listeners
- Updating the DOM manually

Why use it?

To handle things that are **external to the component UI**, such as:

- Calling APIs
- Changing page titles
- Subscribing/unsubscribing to events



Example:

```
useEffect(() => {  
  console.log("Component mounted");  
  fetchUser();  
}, []);
```

✓ With Dependency Array:

```
ts  
  
useEffect(() => {  
  fetchUserById(userId);  
}, [userId]);
```

- Runs only when **userId** changes
- Prevents unnecessary re-fetching

3. Custom Hooks – Reusing Logic

Custom hooks allow you to **extract and reuse logic** across components.

Why use it?

Avoid duplicating code like fetch logic, form handling, or toggling state.

Example:

```
const useApi = (url: string) => {  
  const [data, setData] = useState(null);  
  
  useEffect(() => {  
    fetch(url).then(res => res.json()).then(setData);  
  }, [url]);  
  
  return data;  
};
```

Usage:

```
ts  
  
const userData = useApi("/api/user/1");
```

4. useContext – Sharing State Across Components

React Context lets you share global state without passing props through many levels.

useContext accesses that shared state.

Why use it?

To manage app-wide values like:

- Auth status
- Theme
- User info
- Language/localization

Context setup:

```
interface AuthContextType {  
  user: string;  
  login: (name: string) => void;  
}  
  
const AuthContext = createContext<AuthContextType | undefined>(undefined);
```

Using the context:

```
const { user, login } = useContext(AuthContext)!;
```

Benefit: Clean, no prop-drilling. Just access global state where needed.

5. useReducer – Managing Complex or Structured State

useReducer is useful when:

- State logic is **complex** or depends on **previous values**
- You want a Redux-like structure
- There are **multiple related state values**

Example:

```
type Action = { type: "increment" } | { type: "decrement" };
```

```
const reducer = (state: number, action: Action) => {  
  switch (action.type) {  
    case "increment": return state + 1;  
    case "decrement": return state - 1;  
    default: return state;  
  }  
};
```

```
const [count, dispatch] = useReducer(reducer, 0);
```

Usage:

ts

```
<button onClick={() => dispatch({ type: "increment" })}>+</button>
```

When to use:

- Complex UIs with many interdependent states (e.g., forms, wizards, tabs)
- You want a clear separation of state and logic

Week 3 Project: E-Commerce Product Catalog

Build a functional mini app using everything learned:

- Product listing with filtering by category/price
- Shopping cart with quantity and total
- All product/user/cart data typed with TypeScript interfaces
- Use useState, useContext, and useEffect
- Responsive UI with Tailwind

Week 4: Advanced React Patterns & Form Handling

Goal:

Learn how to write **more reusable, scalable, and performant components**, and handle **complex form logic** with validation, controlled inputs, and third-party libraries like **React Hook Form**.

Day 1–2: Advanced Component Patterns

What is it?

A Higher-Order Component is a **function that takes a component and returns a new component** with enhanced behavior.

Why use it?

- Reuse logic like authentication, tracking, permission checks
- Keep UI code clean while abstracting functionality

Example: Adding authentication to any component

```
const withAuth = (Component) => {  
  return (props) => {  
    const isLoggedIn = checkAuth();  
  
    if (!isLoggedIn) return <Login />;  
    return <Component {...props} />;  
  };  
};  
  
// Usage  
const DashboardWithAuth = withAuth(Dashboard);
```

2. Render Props Pattern

What is it?

Render Props is a pattern where a component takes a function as a prop to determine what it should render.

Why use it?

- Share logic across components without repeating code
- Useful for flexible layouts or data-fetching logic

Example: Sharing data fetch logic

```
const DataProvider = ({ render }) => {
  const [data, setData] = useState([]);

  useEffect(() => {
    fetch("/api/items").then(res => res.json()).then(setData);
  }, []);

  return render(data);
};

// Usage
<DataProvider render={({data}) => <ItemList items={data} /> />
```

3. Compound Components

What is it?

Compound components are a group of **interconnected components** that **share state** and are designed to be used together.

Why use it?

- Provide intuitive and readable structure for grouped UI like modals, tabs, accordions

Example: Custom Modal

```
const Modal = ({ children }) => <div className="modal">{children}</div>;
Modal.Header = ({ children }) => <div className="modal-header">{children}</div>;
Modal.Body = ({ children }) => <div className="modal-body">{children}</div>;

// Usage
<Modal>
  <Modal.Header>Confirm Action</Modal.Header>
  <Modal.Body>Are you sure you want to delete this?</Modal.Body>
</Modal>
```

Day 3–4: Form Handling in React

1. Controlled Components

What is it?

A **controlled component** is a form input that is fully managed by React state.

Why use it?

- Full control of input values
- Easy validation and real-time logic (like disabling submit buttons)

Example:

```
const [name, setName] = useState("");

return (
  <input
    type="text"
    value={name}
    onChange={(e) => setName(e.target.value)}
  />
);
```

2. Validation form (Manual)

Why it matters:

Forms must have validation to ensure correct data. You can validate manually using helper functions.

Example:

```
const validateForm = (data) => {
  const errors = {};
  if (!data.email.includes("@")) errors.email = "Invalid email";
  if (data.password.length < 6) errors.password = "Password too short";
  return errors;
};
```

3. React Hook Form – Easy, Performant Forms

Why use it?

- Less boilerplate than controlled components
- High performance with minimal re-renders
- Built-in validation support

Example:

```
import { useForm } from "react-hook-form";

const { register, handleSubmit, formState: { errors } } = useForm();

const onSubmit = (data) => console.log(data);

return (
  <form onSubmit={handleSubmit(onSubmit)}>
    <input {...register("name", { required: true })} />
    {errors.name && <span>Name is required</span>}

    <input type="email" {...register("email", { required: true })} />
    <button type="submit">Submit</button>
  </form>
);
```

Day 5: Performance Optimization in React

1. React.memo

What is it?

React.memo prevents unnecessary re-renders when props haven't changed.

When to use:

- Reusable presentational components
- Lists of items
- Buttons or inputs in large trees

Example:

```
const MemoizedButton = React.memo(({ label }) => {
  console.log("Rendering:", label);
  return <button>{label}</button>;
});
```

2. useMemo and useCallback

useMemo – caches the result of expensive calculations

Example:

```
const total = useMemo(() => {
  return items.reduce((sum, item) => sum + item.price, 0);
}, [items]);
```

useCallback – memoizes function definitions to prevent re-creation on every render

Example:

```
const handleClick = useCallback(() => {
  console.log("Clicked!");
}, []);
```

End-of-Week Project: User Registration System

Build a full-featured system with:

- Form with inputs: name, email, password, confirm password
- Controlled inputs or React Hook Form
- Validation: required fields, email format, password match
- Error handling and UX messages
- Memoized components for input groups
- Submit button disabled unless form is valid
- Optimized rendering using **React.memo**, **useCallback**, and **useMemo**

Week 5: Redux State Management with Redux Toolkit

Goal:

Learn how to manage **application-wide (global) state** using Redux Toolkit, handle **async data**, and integrate Redux patterns into real React projects using **React-Redux**.

Day 1–2: Redux Fundamentals

What is Redux?

Redux is a **state management library** that helps you manage **application-wide state** in a predictable way. Unlike **useState** or **useContext**, Redux stores your state **in a centralized store**, making it easier to share, update, and debug complex data flows.

Why Use Redux?

- Share state **between distant components** (e.g., auth state, cart items)
- Keep app state **predictable and testable**
- Enable **time-travel debugging** (undo, replay actions)
- Great for **larger applications** with deeply nested components

1. Store Setup with Redux Toolkit

Redux Toolkit (RTK) simplifies Redux setup with prebuilt helpers.

Example:

```
import { configureStore } from '@reduxjs/toolkit';
import userReducer from '../features/userSlice';

export const store = configureStore({
  reducer: {
    user: userReducer
  }
});
```

This store now holds the global user state. You'll use it across your app with **useSelector** and **useDispatch**.

2. Slices: Actions + Reducers in One

Instead of writing actions and reducers separately, Redux Toolkit combines them using **createSlice**.

Example:

```
import { createSlice } from '@reduxjs/toolkit';

const userSlice = createSlice({
  name: 'user',
  initialState: {
    currentUser: null,
    loading: false
  },
  reducers: {
    setUser(state, action) {
      state.currentUser = action.payload;
    },
    logout(state) {
      state.currentUser = null;
    }
  }
});

export const { setUser, logout } = userSlice.actions;
export default userSlice.reducer;
```

This creates:

- A reducer to update state
- Actions like setUser, logout to dispatch from components

4. Selectors: Reading State

Use useSelector to get values from the store.

```
const user = useSelector((state: RootState) => state.user.currentUser);
```

Day 3–4: Advanced Redux Patterns

1. Async Logic with Thunks (createAsyncThunk)

Redux Toolkit has built-in support for async operations like fetching data from APIs.

Example:

```
import { createAsyncThunk } from '@reduxjs/toolkit';

export const fetchUser = createAsyncThunk('user/fetch', async (userId) => {
  const res = await fetch(`/api/users/${userId}`);
  return res.json();
});
```

5.

Automatically dispatches:

- user/fetch/pending
- user/fetch/fulfilled
- user/fetch/rejected

You handle these in your slice to update loading/error state.

2. Middleware: Custom Logging or Access Control

Middleware lets you **intercept actions** before they reach the reducer — great for logging, API token injection, etc.

Example:

```
ts

const logger = (store) => (next) => (action) => {
  console.log('Dispatching:', action.type);
  return next(action);
};
```

Add it to the store:

```
ts

configureStore({
  reducer,
  middleware: (getDefaultMiddleware) => getDefaultMiddleware().concat(logger)
});
```

3. Redux DevTools

DevTools let you:

- Inspect dispatched actions
- View state changes over time
- Rewind/Replay app states

Just open your browser's Redux DevTools extension after setup — works out of the box with `configureStore`.

Day 5: React + Redux Integration Patterns

1. useSelector & useDispatch

- useSelector gets state from the store
- useDispatch sends actions to update it

Example:

```
const dispatch = useDispatch();

<button onClick={() => dispatch(logout())}>Logout</button>
```

2. Type Safety with Redux

Type your store and slices to catch errors early.

Example:

```
interface RootState {
  user: {
    currentUser: User | null;
    loading: boolean;
  };
}
```

Type `useSelector`:

```
ts

const user = useSelector((state: RootState) => state.user.currentUser);
```

End-of-Week Project: Blog Management System with Redux

What to Build:

A full CRUD blog app using Redux for global state

- Authentication state (login/logout)
- **Post Management:** Add, edit, delete posts using Redux
- **Comments:** Store and show comments using slice
- **Reactions:** Like/dislike logic
- All state managed in Redux slices
- Optional: Async actions with createAsyncThunk

Week 6: API Integration & Data Fetching

Goal:

Teach trainees how to:

- Connect React apps to **external APIs**
- Handle **HTTP requests** and **authentication**
- Use tools like **Axios** and **React Query** for efficient data fetching
- Add **real-time data updates** using WebSockets and Server-Sent Events (SSE)

Day 1–2: HTTP Client Setup with Axios

1. What is Axios?

Axios is a promise-based HTTP client for making API calls. It's widely used in React apps because it's:

- Easier to configure than `fetch`
- Supports interceptors for handling tokens and errors
- Automatically parses JSON

2. Axios Base Configuration

Set a global base URL, timeout, and headers using `axios.create` so you don't repeat the setup in every request.

Example:

```
const api = axios.create({
  baseURL: 'https://api.example.com',
  timeout: 5000
});
```

3. Global Error Handling with Interceptors

Interceptors let you add logic before or after a request/response. They're great for:

- Attaching tokens
- Centralized error messages (e.g., toast popups)

Example:

```
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response.status === 401) {
      // auto logout or show auth modal
    }
    return Promise.reject(error);
  }
);
```

4. Authentication with Axios

Automatically attach JWT or session token to each request header.

Example:

```
api.interceptors.request.use(config => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

Day 3–4: React Query (Now TanStack Query)

1. What is React Query?

TanStack Query (React Query) is a powerful data-fetching library that:

- Manages loading, caching, error states
- Handles background syncing, pagination, and refetching
- Is more efficient and scalable than `useEffect` + `axios`

2. Fetching Data with `useQuery`

Example:

```
import { useQuery } from '@tanstack/react-query';

const { data, isLoading, error } = useQuery(['users'], () =>
  api.get('/users').then(res => res.data)
);
```

- 'users' is the cache key
- Automatically handles loading state, errors, and caching

3. Mutations with useMutation

Use useMutation to create, update, or delete data.

```
const mutation = useMutation(addUser, {
  onSuccess: () => {
    queryClient.invalidateQueries(['users']); // Refresh users
  }
});

const addUser = (newUser) => api.post('/users', newUser);
```

4. Caching & Manual Updates

React Query caches your data — this improves speed and user experience. You can also manually update cache after a mutation.

```
queryClient.setQueryData(['user', userId], updatedUserData);
```

Day 5: Real-Time Data Handling

1. WebSockets

WebSockets allow persistent, two-way connections between client and server. Perfect for:

- Chat apps
- Live dashboards
- Multiplayer games

```
const socket = new WebSocket('ws://localhost:8080');

socket.onmessage = (event) => {
  const message = JSON.parse(event.data);
  console.log('New message:', message);
};
```

2. Server-Sent Events (SSE)

SSE allows the server to push updates to the client over HTTP. Great for:

- Notifications
- News feeds
- Live scores

Example

```
const eventSource = new EventSource('/api/notifications');

eventSource.onmessage = (event) => {
  console.log('New event:', JSON.parse(event.data));
};
```

End-of-Week Project: Real-Time Chat App

Build a chat application with:

- User authentication (token-based)
- Message history (GET /messages)
- Send message (POST /messages)
- Real-time chat updates via WebSocket or SSE
- UI State: loading, error, input validation, auto-scroll

Week 7: Testing React Applications

Goal:

Equip learners with the skills to test their React applications at different levels:

- **Unit tests** for isolated logic
- **Integration tests** for component interactions
- **End-to-End (E2E) tests** to simulate full user journeys

They'll use tools like **Jest**, **React Testing Library**, and **Cypress** for comprehensive test coverage.

Day 1–2: Unit Testing Fundamentals

1. What is Unit Testing?

Unit testing is the process of testing **individual pieces of code in isolation**, such as a single function or a small component.

2. Setting Up Jest and React Testing Library

- **Jest** is the default test runner for React apps (especially with Create React App).
- **React Testing Library** (RTL) helps test components by simulating how users interact with them, not by checking implementation details.

Example:

```
import { render, screen } from '@testing-library/react';
import Button from './Button';

test('renders button with text', () => {
  render(<Button label="Click me" />);
  expect(screen.getByRole('button')).toHaveTextContent('Click me');
});
```

Goal: Validate if the button renders correctly for the user.

3. Testing Component Events and Props

You can simulate user interactions like clicks and check if the expected callback or UI change occurs.

Example:

```
import { fireEvent } from '@testing-library/react';

test('calls onClick when button is clicked', () => {
  const mockFn = jest.fn();
  render(<Button label="Submit" onClick={mockFn} />);
  fireEvent.click(screen.getByText('Submit'));
  expect(mockFn).toHaveBeenCalledTimes(1);
});
```

4. Mocking API Calls

In unit tests, you should mock any external dependency (like API calls) so tests don't rely on the real backend.

Example:

```
jest.mock('../api', () => ({
  fetchUsers: jest.fn(() => Promise.resolve([{ id: 1, name: 'John' }])),
}));
```

This allows you to test components without needing actual API responses.

Day 3–4: Advanced Testing Techniques

1. Integration Testing

Integration tests check how multiple components or modules work together.

Example: Test a form with submit logic

```
test('submits form and resets input', async () => {
  render(<LoginForm />);
  fireEvent.change(screen.getByPlaceholderText('Email'), {
    target: { value: 'test@example.com' }
  });
  fireEvent.click(screen.getByRole('button', { name: 'Submit' }));

  expect(await screen.findByText(/welcome/i)).toBeInTheDocument();
});
```

2. Testing Custom Hooks

Hooks hold logic, so test them independently with `@testing-library/react-hooks` or directly if simple.

Example:

```
import { renderHook, act } from '@testing-library/react-hooks';
import useCounter from './useCounter';

test('increments counter', () => {
  const { result } = renderHook(() => useCounter());
  act(() => result.current.increment());
  expect(result.current.count).toBe(1);
});
```

3. Testing React Context

When a component uses `useContext`, wrap it in the appropriate provider before testing.

Example:

```
render(
  <AuthContext.Provider value={{ user: { name: 'Jane' } }}>
    <Dashboard />
  </AuthContext.Provider>
);
expect(screen.getByText('Welcome, Jane')).toBeInTheDocument();
```

Day 5: End-to-End (E2E) Testing with Cypress

What is E2E Testing?

E2E tests simulate a real user journey — clicking through pages, submitting forms, verifying outcomes — as if a user were manually testing the app in the browser

1. Setting Up Cypress

Cypress is an all-in-one E2E testing tool that runs in the browser and provides a live UI for watching your tests.

```
npm install cypress
npx cypress open
```

2. Writing a Cypress Test

Example: Login Flow

```
describe('Login Flow', () => {  
  it('logs in a user successfully', () => {  
    cy.visit('/login');  
    cy.get('[data-cy=email]').type('user@example.com');  
    cy.get('[data-cy=password]').type('password123');  
    cy.get('[data-cy=submit]').click();  
  
    cy.contains('Welcome back, user').should('be.visible');  
  });  
});
```

Best practice: Use data-cy attributes for targeting elements in Cypress.

Project: Add Comprehensive Tests

Your task is to revisit a previous project (e.g., Blog App or Chat App) and implement:

-  **Unit tests** for all UI components
-  **Integration tests** for user actions and state changes
-  **E2E tests** for critical user journeys

Coverage Goal: Aim for **80%+ test coverage** using tools like Jest's built-in coverage report:

Week 8: CI/CD & Deployment Pipeline

Goal:

Help trainees automate the process of:

- Running tests,
- Building applications,
- Deploying apps to production environments.

This ensures faster delivery, fewer bugs, and better collaboration.

Day 1–2: GitHub Actions for CI/CD

1. What is CI/CD?

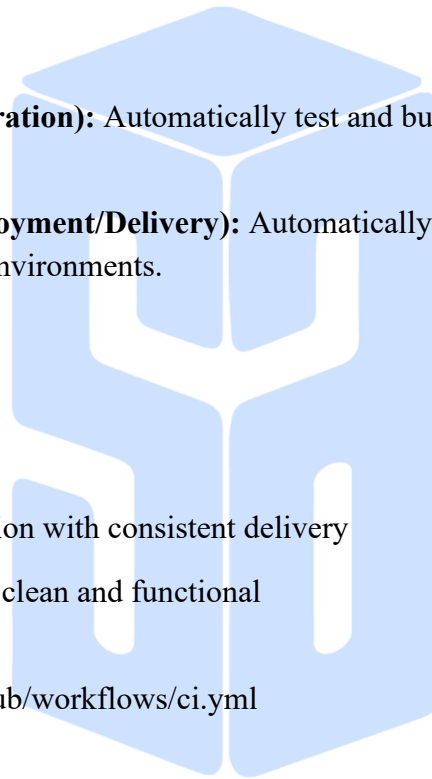
- **CI (Continuous Integration):** Automatically test and build your app every time code is pushed.
- **CD (Continuous Deployment/Delivery):** Automatically deploy tested code to production or staging environments.

2. Why it matters:

- Reduces manual errors
- Saves time
- Ensure team collaboration with consistent delivery
- Keeps production code clean and functional

3. Workflow Configuration

Create a workflow file in `.github/workflows/ci.yml`



Example:

```
yaml
name: CI Pipeline

on:
  push:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3
      - name: Install dependencies
        run: npm install
      - name: Run tests
        run: npm test -- --coverage --watchAll=false
      - name: Build app
        run: npm run build
```

This ensures your app is automatically tested and built every time code is pushed to the **main** branch.

Day 3–4: Deployment Strategies

1. Netlify – Static Site Deployment

Best for: Static React apps (e.g., frontend only, no backend APIs)

How to deploy:

- Push code to GitHub
- Connect your GitHub repo to Netlify
- Netlify auto-builds and deploys on every push

Free SSL, custom domain, CI/CD built-in

2. Vercel – Serverless Hosting

Best for: React (Next.js), TypeScript apps, or apps with dynamic APIs

Command-Line Deployment:

```
npm install -g vercel
```

```
vercel --prod
```

Vercel pulls environment variables from `.env`, supports previews, and auto-deploys from GitHub too.

Docker – Containerized Deployment

Best for: Backend apps, full-stack apps, and scalable infrastructure

Dockerfile Example:

```
Dockerfile

FROM node:16-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
CMD ["npm", "run", "build"]
```

Docker allows you to package your entire app with its environment and dependencies, ensuring consistent behavior across machines.

Day 5: Environment Management, Feature Flags & Monitoring

1. Environment Variables

React apps often need different settings for development, staging, and production.

Example:

```
REACT_APP_API_URL=https://api.example.com
REACT_APP_ENV=production
```

Place variables in `.env`, `.env.production`, or `.env.local`.

Use them in code like:

```
const baseURL = process.env.REACT_APP_API_URL;
```

2. Feature Flags

Feature flags help you control which features are visible in production without changing your code.

Example:

```
if (process.env.REACT_APP_FEATURE_CHAT === 'true') {  
  return <LiveChat />;  
}
```

End-of-Week Assignment: Automate and Deploy:

Project Goals:

Build a CI/CD pipeline that:

1. Runs unit and integration tests
2. Builds the React app
3. Deploys to Netlify or Vercel
4. Uses .env for production API
5. Implements feature flags for showing/hiding components
6. Adds error tracking (e.g., with Sentry)

Week 9: Advanced Topics & Capstone Project

Focus Areas:

- Server-side rendering (SSR) with Next.js
- AI integration using TensorFlow.js
- Real-world performance optimization
- Security practices for modern apps
- Development of a full-featured capstone project

Day 1: Server-Side Rendering (SSR) with Next.js

Why SSR?

Server-Side Rendering improves initial load performance, SEO, and dynamic data handling by rendering HTML on the server instead of waiting for the browser to load JavaScript.

Key Concepts:

Setting up Next.js:

```
npx create-next-app@latest my-app --typescript
```

This scaffolds a ready-to-use React + TypeScript app with SSR capabilities.

getServerSideProps – Fetch data on every request

```
export async function getServerSideProps(context) {  
  const res = await fetch('https://api.example.com/posts');  
  const data = await res.json();  
  return { props: { data } };  
}
```

Best for: User-authenticated pages, search results, or live data.

getStaticProps + getStaticPaths – Pre-generate pages at build time

```
export async function getStaticProps() {  
  const posts = await fetchPosts();  
  return { props: { posts }, revalidate: 60 };  
}
```

Day 2: AI Integration with TensorFlow.js

Why AI in React?

AI enables intelligent features such as image recognition, text analysis, voice-to-text, and recommendations — enhancing app interactivity and user value.

Setting Up TensorFlow.js

```
npm install @tensorflow/tfjs @tensorflow-models/mobilenet
```

Day 3: AI + SSR Mini Project

Mini Project: “Smart Image Analyzer”

Build an SSR-powered React app using Next.js and TensorFlow.js to analyze images in real-time.

Features:

- Drag-and-drop image upload
- AI detection using COCO-SSD
- Display detected objects with labels and confidence scores
- SSR-enabled results page for SEO and performance

Example Code Snippet:

```
const [predictions, setPredictions] = useState([]);

const analyzeImage = async (imgElement: HTMLImageElement) => {
  const model = await cocoSsd.load();
  const results = await model.detect(imgElement);
  setPredictions(results);
};
```

Day 4: Performance Optimization & Security

Performance Best Practices:

1. Code Splitting with React.lazy

```
const AIComponent = React.lazy(() => import('./AIComponent'));
```

Loads components only when needed to reduce initial bundle size.

2. Bundle Analysis

```
npm run build && ANALYZE=true next build
```

Check what increases bundle size — TensorFlow models can be large.

3. Service Workers / Caching (PWA)

- Use Workbox to cache models
- Allow offline AI inference

Security Best Practices:

- **XSS Prevention**
Use libraries like DOMPurify to sanitize user-uploaded data (text or images).
- **CSRF Protection**
Include secure tokens in AI API calls, especially when processing user input on the server.
- **Content Security Policy (CSP)**
Set up secure headers to whitelist trusted domains (e.g., TensorFlow CDN).

Day 5: Individual Capstone Project

Project Title:

“AI-Powered Portfolio Dashboard with SSR”

Each trainee builds a portfolio-quality full-stack React app with integrated AI features and server-side rendering.

Key Requirements:

- **SSR setup** using Next.js
- **AI feature** (choose one below)
- **User authentication** (JWT-based)
- **Real-time notifications** (WebSocket or polling)
- **File uploads** + AI model processing
- **Advanced filtering, search, and sorting**
- **Fully responsive mobile-first design**
- **CI/CD pipeline** (e.g., GitHub Actions + Vercel)
- **Full test coverage**

Choose One AI Feature:

1. **Smart Image Gallery** – Auto-tagging and categorization
2. **Content Recommender** – Personalized article/video suggestions
3. **Text Analytics** – Sentiment analysis + keyword extraction
4. **Voice-to-Text Notes** – Speech input converted to editable notes

Core Tech Stack:

- **Next.js + TypeScript**
- **Redux or Context API**
- **React Query** for data fetching
- **Tailwind CSS**
- **TensorFlow.js** for AI
- **Cypress + Jest** for testing